Mesh-Based Self-Collision

Vertex-Face Proximity on GPU

P(F) Shape Matching PBD | Replaces Particle-Based Approach

1. Why Particle-Based Self-Collision Failed

The particle-based approach treats each vertex as a sphere of radius 0.012. Collision triggers when two sphere centers are within 2r = 0.024. However, surface triangles can interpenetrate even when all vertices are separated by more than 0.024. Two triangular faces can pass through each other while their vertices remain far apart.

Particle vs Mesh Collision

Aspect	Particle-Based (old)
Detects	Vertex-vertex overlap only
Misses	Face-face interpenetration
Suitable for	Granular/SPH, not continuous mesh

Aspect	Mesh-Based (new)
Detects	Vertex passing through face
Handles	Surface self-intersection
Suitable for	Deformable tet mesh (our case)

2. Approach: Vertex-Face Proximity + Spatial Hash Grid

Core Algorithm

1. Hash surface face centroids into spatial grid (reuse CUB radix sort pattern)
2. For each surface vertex, query 3x3x3 neighboring cells to find candidate faces
3. Skip topologically adjacent faces (vertex is part of face, or 1-ring neighbor)
4. Compute point-triangle distance; if < thickness, apply PBD correction
5. Corrections accumulated into d_deltas via atomicAdd (same as shape matching)

Modified Jacobi Loop

```
for iter in [0, num_iterations):
    clear_deltas_kernel
    cluster_solve_kernel              // shape matching constraints
    if self_collision_enabled:
        build_face_hash_grid()        // hash face centroids (CUB sort)
        mesh_self_collision_kernel    // vertex vs non-adjacent faces
    apply_kernel                      // merge shape matching + collision
    collision_kernel                  // floor + knife hard constraints
```

3. New GPU Data Structures

Surface Mesh (shape_matching_gpu.h)

```
struct SurfaceMeshGPU {
    int* face_a;              // [max_faces] triangle vertex A
    int* face_b;              // [max_faces] triangle vertex B
    int* face_c;              // [max_faces] triangle vertex C
    int* vertex_is_surface;   // [N] 1 if surface vertex, 0 otherwise
    int num_faces;            // current face count
    int max_faces;            // allocated capacity
};
```

Vertex Adjacency CSR (for topological exclusion)

```
int* d_neighbor_starts;   // [N+1] CSR row pointers
int* d_neighbor_ids;      // [total_entries] sorted neighbor IDs
// Built once from tet_mesh.vertex_neighbors (topology fixed)
```

Face Hash Grid (reuses CUB sort pattern)

```
unsigned int* d_face_hash, d_face_index;         // unsorted
unsigned int* d_face_hash_sorted, d_face_idx_sorted; // sorted
unsigned int* d_face_cell_start, d_face_cell_end;    // [TABLE_SIZE]
void* d_face_cub_temp;
```

4. CUDA Kernels

4.1 compute_face_hash_kernel

Per face: compute centroid, hash into grid cell.

```
__global__ void compute_face_hash_kernel(
    float3* positions, int* fa, int* fb, int* fc,
    unsigned int* hash, unsigned int* index,
    float cell_size, int table_size, int num_faces) {
    int f = blockIdx.x * blockDim.x + threadIdx.x;
    if (f >= num_faces) return;
    float3 va=positions[fa[f]], vb=positions[fb[f]], vc=positions[fc[f]];
    float cx=(va.x+vb.x+vc.x)/3, cy=(va.y+vb.y+vc.y)/3, cz=(va.z+vb.z+vc.z)/3;
    int ix=floor(cx/cell_size), iy=floor(cy/cell_size), iz=floor(cz/cell_size);
    hash[f] = (ix*73856093 ^ iy*19349663 ^ iz*83492791) & (table_size-1);
    index[f] = f;
}
```

4.2 point_triangle_distance (device function)

Standard closest-point-on-triangle algorithm. Projects point onto triangle plane, computes barycentric coordinates. If projection is inside triangle, distance = point-to-plane. Otherwise, clamps to nearest edge or vertex. Returns distance + barycentric coords for correction distribution.

```
__device__ float point_triangle_distance(
    float3 p, float3 a, float3 b, float3 c,
    float3* closest, float* u, float* v, float* w) {
    // 1. Compute edges and project
    float3 ab=b-a, ac=c-a, ap=p-a;
    float d1=dot(ab,ap), d2=dot(ac,ap);
    float d3=dot(ab,ab), d4=dot(ab,ac), d5=dot(ac,ac);
    // 2. Barycentric coords via Cramer's rule
    // 3. Clamp to triangle edges if outside
    // 4. Return distance to closest point
    // (~40 lines, well-known algorithm)
}
```

4.3 is_neighbor (device function)

Binary search in sorted CSR neighbor list for topological exclusion.

```
__device__ bool is_neighbor(int v, int u,
    int* starts, int* ids) {
    int lo = starts[v], hi = starts[v+1];
    while (lo < hi) {  // binary search
        int mid = (lo + hi) / 2;
        if (ids[mid] == u) return true;
        if (ids[mid] < u) lo = mid+1; else hi = mid;
    }
    return false;
}
```

4.4 mesh_self_collision_kernel

Per surface vertex: query 3x3x3 neighbor cells, iterate candidate faces, skip adjacent faces, compute point-triangle distance, apply PBD correction.

```
__global__ void mesh_self_collision_kernel(
    float3* pred_pos, int* removed, int* fixed, int* is_surface,
    int* face_a, int* face_b, int* face_c,
    int* nbr_starts, int* nbr_ids,
    uint* face_sorted_idx, uint* face_cell_start, uint* face_cell_end,
    float3* deltas, int* delta_counts,
    float thickness, float stiffness, float cell_size,
    int table_size, int N) {
    int v = blockIdx.x * blockDim.x + threadIdx.x;
    if (v >= N || removed[v] || !is_surface[v]) return;

    float3 p = pred_pos[v];
    // Query 3x3x3 neighbor cells
    for (dx,dy,dz in [-1,0,1]^3):
        h = hash(cell + offset)
        for each face f in cell:
            a,b,c = face vertices
            if (v==a || v==b || v==c) continue;  // skip own face
            if (is_neighbor(v,a) || is_neighbor(v,b) || is_neighbor(v,c))
                continue;  // skip 1-ring adjacent
            dist = point_triangle_distance(p, pos[a], pos[b], pos[c])
            if (dist < thickness):
                correction along face normal
                atomicAdd(deltas[v],  +0.25 * corr)  // vertex
                atomicAdd(deltas[a],  -0.75*u * corr) // face verts
                atomicAdd(deltas[b],  -0.75*v * corr)
                atomicAdd(deltas[c],  -0.75*w * corr)
}
```

5. PBD Correction Distribution

When vertex V is within thickness of face (A,B,C), 4 particles are involved. Correction is split: 1/4 to V (pushed away), 3/4 to face vertices (pushed back), weighted by barycentric coordinates of the closest point on the face.

```
// Vertex V: pushed along +normal
deltas[V] += 0.25 * (thickness - dist) * stiffness * normal

// Face vertices: pushed along -normal, weighted by barycentric
deltas[A] -= 0.75 * bary_u * (thickness - dist) * stiffness * normal
deltas[B] -= 0.75 * bary_v * (thickness - dist) * stiffness * normal
deltas[C] -= 0.75 * bary_w * (thickness - dist) * stiffness * normal
```

6. Dynamic Surface Synchronization

Surface faces change when particles are removed (debulking). The CPU already calls rebuild_surface_faces() after removal. After this, call gpu_solver.upload_surface_mesh() to re-upload face_a/b/c and vertex_is_surface to GPU. Vertex neighbors (tet topology) never change, uploaded once at init.

```
// In main_gpu_viewer.cpp, after rebuild_surface_faces():
gpu_solver.upload_surface_mesh(
    tet_mesh.surface_faces,    // new face list
    N);                        // rebuild vertex_is_surface
```

7. Topological Exclusion

A surface vertex must NOT collide with faces it is part of or topologically adjacent to. Without exclusion, every vertex would collide with its own faces, causing explosion.

1. Skip if vertex V is a vertex of face (A,B,C): V==A || V==B || V==C
2. Skip if V is a 1-ring mesh neighbor of any face vertex: is_neighbor(V,A) || is_neighbor(V,B) || is_neighbor(V,C)

The is_neighbor() check uses binary search in the sorted CSR adjacency list. vertex_neighbors is built from tet connectivity (each tet creates 6 edges), giving ~12-14 neighbors per vertex. Binary search cost: ~4 comparisons.

8. Performance Budget

Operation	Per Iteration	4 Iter/Step
face hash + CUB sort	0.07 ms	0.28 ms
mesh_collision (5k verts)	0.3-0.5 ms	1.2-2.0 ms
Total	**~0.5 ms**	**~1.5-2.0 ms**

Current step: ~2ms. With mesh collision: ~4ms. Viewer: ~80 FPS (acceptable). Optimization: run only on iterations 0 and 2, halving cost to ~1ms/step.

9. Configuration Parameters

```
gpu_solver.config.self_collision_thickness = 0.01f;  // collision thickness
gpu_solver.config.self_collision_stiffness = 0.5f;    // PBD stiffness
gpu_solver.config.self_collision_enabled = 1;          // toggle
```

thickness = 0.01: less than minimum non-adjacent face distance (~0.06 for cube32), so rest state does not trigger collision. Only activates under deformation.

10. Files to Modify

File	Changes
solver/shape_matching_gpu.h	SurfaceMeshGPU struct + neighbor CSR + methods
solver/shape_matching_gpu.cu	4 kernels + device funcs + hash grid + step()
main_gpu_viewer.cpp	init_surface_mesh() + upload after rebuild
core/tet_mesh.h	Optional: flatten_neighbors_csr() helper

11. Implementation Sequence

1. Add SurfaceMeshGPU struct and member variables to shape_matching_gpu.h
2. Implement init_surface_mesh(): allocate GPU buffers, upload initial faces + neighbors CSR
3. Implement upload_surface_mesh(): re-upload faces after removal
4. Implement compute_face_hash_kernel + build_face_hash_grid() (copy CUB pattern)
5. Implement point_triangle_distance device function (~40 lines)

6. Implement is_neighbor device function (binary search, ~10 lines)
7. Implement mesh_self_collision_kernel (~80 lines)
8. Replace old particle self-collision call in step() Jacobi loop
9. Hook up upload_surface_mesh() in main_gpu_viewer.cpp after rebuild_surface_faces()
10. Add cleanup for new GPU allocations

12. Verification

1. Rest state: collision does NOT trigger (thickness < min non-adjacent face distance)
2. Knife push: surface does not self-intersect when deformed
3. Debulking: exposed interior surface does not intersect
4. Performance: step time < 5ms with self-collision enabled
5. Stability: no oscillation between shape matching and collision corrections

13. Future Enhancements

- Edge-edge collision: detect when two non-adjacent surface edges cross
- Continuous collision detection (CCD): prevent tunneling at high velocities
- 2-ring topological exclusion: for very coarse meshes